

Rendez-vous

One room, one agent, one artifact — many humans, on the surfaces they already use.

THE WEEK

The sandbox is now a commodity.

- OpenAI's Agents API: nine sandbox partners, one API call (2026-09-10).
- Every one of them is single-principal — one key, one developer, one session.

The uncontested ground isn't the sandbox. It's the room around it.

THE PRIMITIVE

Room

- Code — RDV-7F3K, a durable handle
- Members — many humans, many surfaces
- Tiers — messenger, email, room-web
- One session — the one agent turn-loop
- One artifact URL — stable, public, live

THE FIDELITY LADDER

Three tiers, one room

TIER	SURFACE	SENDS	RECEIVES	LATENCY
1	WhatsApp / Telegram	plain text turns	final reply + artifact URL, terse	seconds
2	Email	reply-to-thread turns	per-turn digest + artifact link	minutes
3	Laptop web room	turns, permission answers	full transcript + live artifact iframe	live

Telegram and laptop are already two real surfaces.

RUN 3 — WHAT ACTUALLY HAPPENED

A real phone, a real operator, one artifact

- 0s** Alice's "new" on real Telegram — room created
- 86.5s** Cold boot — session + artifact ready (4th and final boot budget)
- +~20s** Bob joins from the laptop room view
- live** Attribution confirmed in the stream: [Bob · room-web], [6371794295 · messenger]
- ~33s** Agent finds and edits the artifact, unaided

Curl-verified at every step. Delivery to the phone: operator-confirmed.

One shape, five independent bugs

1

FOUND LIVE, TONIGHT

the liveness check reads `res.ok`, not `status` — a killed session still answers 200, so resume reports success in 0.7s having resumed nothing

2

queue: `true` omitted → a mid-turn prompt is silently dropped

3

the resume cursor stays frozen → every reply after a resume vanishes

4

an account-pinned inbound route → never fires, filtered out silently

5

the agent proposes to rebuild a Gmail capability it already held

The first four lost data quietly. The fifth reported success while doing nothing — the correct field was already parsed one function over.

None crash. All look like working code or correct config. This is evidence you only get by running it.

A DESIGNED POSITION

Credentials are per-member, not per-room

- A shared room holding one key shares it with everyone who has the code.
- Our position: each member attaches their own tools over rendezvous — nothing inherited.
- Revocation is an unpair, not a key rotation — there was never a shared secret to rotate.

WHY A ROOM

Why a room, not two people and one bot

- 1 the agent asks Alice for the product shot
- 2 the agent asks Bob for the one-line positioning
- 3 each answers on their own channel, in their own time
- 4 the agent synthesizes both into the deck — and it goes to the client as a PDF

The agent addresses each member individually and reconciles what they contribute — not a group chat with a bot bolted on.

Specced, not yet built.

The swap needs a login first

- Tested live: the boot failed at the auth gate, before the adapter swap could even be observed.
- `installAdapters` installs the codex binary — not a login. Nothing is sent to a box automatically.

```
agent_start({ adapter: "codex", model: "gpt-5.2-codex", ... })
```

```
→ 500 sandbox_proxy_failed
```

```
"no codex login found — run `codex login`"
```

```
box destroyed on failure — boots used: 1 of 1
```

Same box, same seed, same port with a different CLI is plausible from source, but unobserved. The real second-brain work is the credential model (§9.3, device-auth) — not the adapter swap.

`docs/CODEX-FLIP.md` — variant A only; B and C untested (1-boot budget).

IS THIS A CHEAT?

Built on our own open-source runtime

- Unmodified. Installed from npm. No fork, no vendored copy.
- The room layer is every line that's new — the judged artifact.
- 8 upstream findings, shipped as a maintainer write-up (docs/UPSTREAM.md).

What was simulated, what was observed

- Runs 1–2: simulated inbound, no phone, no agentpush workspace. Run 3: real Telegram + web, live.
- Paused e2b boxes expire in 20–60 min — and tonight it happened live: box i7jos6lixgkcfrekmi1vl expired mid-rehearsal, and the room kept advertising its now-dead artifact URL.
- Resume-after-kill and the dead artifact link: fixed after rehearsal, re-proof pending. A room-scoped, proxied artifact URL is now being built — ahead of whisper and multimodal.
- We share the default workspace's Telegram routing — its existing responder route can fire alongside ours on the same message, risking a double reply.
- WhatsApp: not provisioned. No number, no key.
- Phase 2 pairing-back: untested — box-to-broker egress, which side mints the token, post-pairing scope.

Work leaves the room — on purpose

- 1 Jeremy asks the room to build a presentation, then reviews it live at the artifact URL
- 2 He asks for a PDF
- 3 **THE CONFIRMATION GATE**
the room previews it first — recipient, channel, subject, the rendered artifact — and sends only when a member explicitly confirms
- 4 the PDF reaches Jeremy's own messenger, and by email, a client who is not a member of the room
- 5 every send lands in the shared transcript: who asked, who confirmed, what was sent, to whom

Every other capability keeps work inside the room. This one lets the room produce a work product that leaves it — the confirmation gate is a room with a notion of authority, not just relay.

Chains fan-in, the artifact, canvakit and agentpush end to end. Being built, not yet observed.

The room is the primitive.

The runtime is a detail.